

Python

```
while True:
    # Check which neighboring spaces adjacent to
    # the mark have not been visited already:
    unvisitedNeighbors = []
    if y > 1 and (x, y - 2) not in hasVisited:
        unvisitedNeighbors.append(NORTH)

    if y < HEIGHT - 2 and (x, y + 2) not in hasVisited:
        unvisitedNeighbors.append(SOUTH)

    if x > 1 and (x - 2, y) not in hasVisited:
        unvisitedNeighbors.append(WEST)

    if x < WIDTH - 2 and (x + 2, y) not in hasVisited:
        unvisitedNeighbors.append(EAST)
```

The JavaScript code continues as follows:

```
while (true) {
    // Check which neighboring spaces adjacent to
    // the mark have not been visited already:
    let unvisitedNeighbors = [];
    if (y > 1 && !JSON.stringify(hasVisited).includes(JSON.stringify([x, y -
2]))) {
        unvisitedNeighbors.push(NORTH);
    }
    if (y < HEIGHT - 2 && !JSON.stringify(hasVisited).includes(JSON.stringify
([x, y + 2]))) {
        unvisitedNeighbors.push(SOUTH);
    }
    if (x > 1 && !JSON.stringify(hasVisited).includes(JSON.stringify([x - 2,
y]))) {
        unvisitedNeighbors.push(WEST);
    }
    if (x < WIDTH - 2 && !JSON.stringify(hasVisited).includes(JSON.stringify
([x + 2, y]))) {
        unvisitedNeighbors.push(EAST);
    }
}
```

The `while` loop continues to loop as long as unvisited neighbors remain for this location in the maze. We create a list or array of unvisited neighboring